МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ПРАКТИЧНА РОБОТА № 9.2

IoT Edge Computing + Cloud Integration з Docker

Варіант 1. Smart Factory Monitoring System

Дисципліна: Технології IoT та SMART-технології

Тема: інтеграція IoT з Edge Computing та Cloud

Відомості	Значення
Студент	Свидан Андрій Андрійович
Група	_____
Навчальний рік	2025/2026
Викладач	О.А. Клименко

Мета: розробити систему IoT моніторингу обладнання з локальною обробкою даних, MQTT-комунікацією, контейнеризацією, хмарною аналітикою та вебвізуалізацією.

Звіт підготовлено на основі реалізованого коду, 8 виконаних модульних тестів і локальної перевірки алгоритмів. Повний запуск Docker Compose та скриншоти інтеграції потрібно підтвердити на комп'ютері студента.

1. Мета роботи та короткий огляд

Розроблено навчальну систему моніторингу промислової лінії. Симулятор періодично формує показники обладнання. Обчислювальний вузол Edge аналізує дані та відразу реагує на перевищення допустимих значень. Cloud накопичує історію, агрегує показники, надає REST API та вебпанель.

Основні завдання

- Сформувати Docker Compose проєкт з шести взаємопов'язаних сервісів.
- Генерувати показники шести IoT сенсорів із добовими коливаннями та несправностями.
- Реалізувати MQTT-передавання, локальні правила Edge та автоматичні дії.
- Забезпечити локальне зберігання і повторне передавання непідтверджених записів.
- Додати Cloud Connector з InfluxDB, агрегуванням та REST API.
- Розробити вебпанель з оновленням показників, графіком та віддаленими командами.

Обґрунтування вибору

Варіант Smart Factory Monitoring System логічно продовжує практичну роботу 9.1, де аналізувалися вібрація, температура підшипників, струм, оберти, шум і тиск. Для швидкого локального реагування використано правила з пороговими значеннями. На відміну від ML, такі правила легко перевірити й пояснити на захисті.

Результат реалізації

Критерій	Виконано
Програмні компоненти	6 Docker-сервісів у конфігурації
IoT-показники	6 типів сенсорів
Комунікація	MQTT QoS 1, HTTP REST
Edge автономність	SQLite-черга та повторна синхронізація
Cloud	InfluxDB, Flask, SQLite-журнал
Візуалізація	HTML, CSS, JavaScript, SVG
Локальні тести	8 із 8 пройдено

2. Архітектура системи

Використано розділення на Edge та Cloud. Сервіси розміщені у двох Docker bridge-мережах. MQTT брокер та InfluxDB доступні в обох мережах. Симулятор і Edge працюють у сегменті edge-net, а Cloud і вебпанель у сегменті cloud-net.

Вузол	Дані / функції	Рівень
Sensor Simulator	Формування 6 вимірювань	Edge
MQTT Broker	Маршрутизація повідомлень	Міст
Edge Processor	Пороги, аварійні дії, буфер	Edge
InfluxDB	Часові ряди edge та cloud	Спільна БД
Cloud Connector	Підтвердження, статистика, API	Cloud
Dashboard	Картки, графік, журнал, команди	Cloud

Схема передавання даних

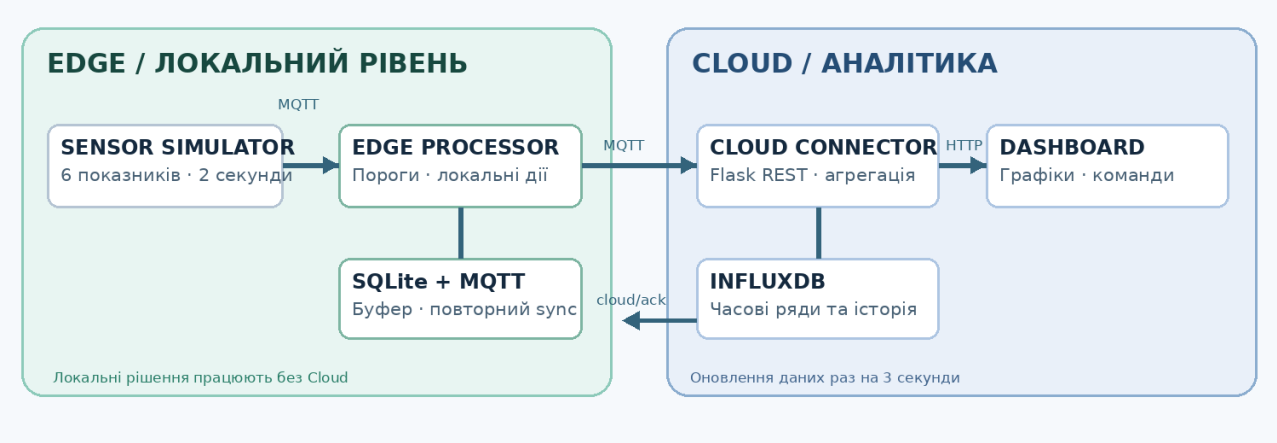


Рисунок 1. Архітектура двох рівнів. MQTT передає дані та команди, Edge має локальний буфер.

Ключовий принцип Edge Computing

При критичній температурі чи вібрації Edge самостійно виконує потрібну дію. Для локального рішення йому не потрібна відповідь від Cloud. Це зменшує залежність аварійного реагування від мережі. Cloud використовується для довготривалої аналітики та контролю з браузера.

3. Docker Environment Setup

Конфігурацію системи описано в `docker-compose.yml`. Кожен Python-компонент отримав окремий Dockerfile із базовим образом Python 3.11 slim. Для вебпанелі використано Nginx, для брокера Eclipse Mosquitto 2.0, для часових рядів InfluxDB 2.7.

Основні параметри

Параметр	Рішення
Порти	8080 для вебпанелі, 8086 для InfluxDB, 1883 для MQTT
Безпека демонстрації	Зовнішні порти прив'язані до 127.0.0.1
Налаштування	.env із параметрами InfluxDB та симулятора
Мережі	edge-net та cloud-net
Volumes	mqtt-data, mqtt-logs, edge-data, cloud-data, influx-data, influx-config
Автозапуск	restart: unless-stopped

Команди запуску на macOS

```
docker --version

docker compose version

docker compose up --build -d

docker compose ps
```

У разі успішного запуску команда `docker compose ps` покаже шість активних сервісів. У цій робочій середовищі Docker Engine відсутній, тому стан контейнерів і мережеву інтеграцію необхідно підтвердити на Mac після запуску Docker Desktop.

Перевірка журналів

```
docker compose logs --tail=20 sensor-simulator

docker compose logs --tail=20 edge-processor

docker compose logs --tail=20 cloud-connector
```

Зупинка сервісів виконується командою `docker compose down`. Існуючі дані у Docker volumes залишаються. Команда `docker compose down -v` видаляє також постійне сховище.

4. Реалізація IoT симулятора

Файли `sensors/patterns.py` і `sensors/simulator.py` генерують і надсилають дані в тему `sensors/factory/telemetry`. Інтервал публікації визначає параметр `SENSOR_INTERVAL`, типовий інтервал для демонстрації становить 2 секунди.

Параметри обладнання

Сенсор	Одиниця	Нормальний сценарій
vibration	мм/с	Вібрація залежить від навантаження
bearing_temp	°C	Температура змінюється за добовим циклом
motor_current	А	Струм корелює з навантаженням
rpm	об/хв	Оберти зі випадковим відхиленням
noise_db	дБ	Акустичний шум змінюється з навантаженням
pressure	бар	Тиск змінюється в невеликому діапазоні

Генерація несправностей

Параметр `FAULT_RATE=0.10` задає ймовірність штучного збою для одного пакета. Передбачено шість категорій несправностей: перегрів, надмірна вібрація, зменшення обертів, низький тиск, перевантаження струмом і шум. Цільові показники виходять за критичні пороги, що дозволяє перевірити локальні правила.

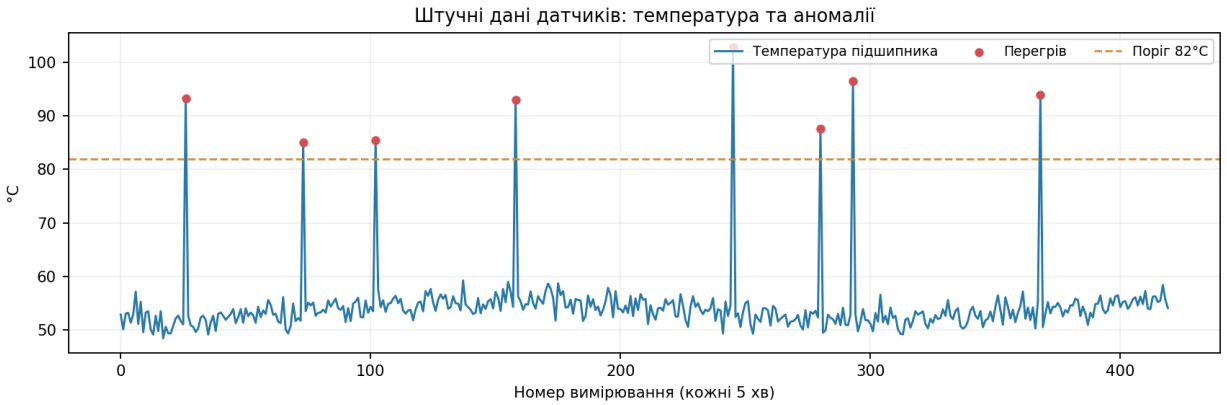


Рисунок 2. Приклад 420 штучних вимірювань із локального тесту. Це перевірка функції генерації даних, не скриншот Docker.

5. Edge Processor та правила реагування

Edge підписується на MQTT-теми `sensors/factory/telemetry`, `cloud/ack` і `cloud/commands`. Кожне вимірювання проходить перевірку порогів із файлу `edge/config.json`. При порушенні правил створюється повідомлення і публікується дія у `actuators/factory`.

Таблиця критичних значень

Показник	Умова	Дія	Рівень
Температура	> 82 °C	cooling_on	critical
Вібрація	> 3.2 мм/с	stop_line	critical
Струм	> 19 A	reduce_load	warning
Оберти	< 900 об/хв	inspect_motor	warning
Шум	> 87 дБ	inspect_noise	warning
Тиск	< 2.4 бар	inspect_pressure	critical

Алгоритм локального рішення

Функція `evaluate(values, thresholds)` перевіряє кожен показник. Вона повертає список аномалій із назвою сенсора, фактичним значенням, порогом, пріоритетом і дією. Додатково Edge аналізує різку зміну температури порівняно з попередніми даними в буфері останніх 120 вимірювань.

Автоматизовані дії

При спрацюванні правила публікується MQTT-команда. `stop_line` переводить стан програмної моделі обладнання в режим зупинки. `cooling_on` і `cooling_off` передають керуючі команди як симуляцію актуаторів. Для реального обладнання потрібні контролери, підтвердження стану та безпечна апаратна інтеграція.

Оцінювання швидкодії

Вимірюється час виконання локального правила, починаючи від надходження повідомлення до формування рішення. Окремо працюють запис у SQLite, MQTT і InfluxDB. Отже, цей показник не дорівнює повній мережевій затримці від датчика до Cloud.

6. Локальна автономність та відмовостійкість

Для захисту від втрати даних Edge використовує SQLite. Кожен пакет з унікальним `event_id` додається до таблиці `pending` перед надсиланням до Cloud. Непідтверджені записи залишаються на диску у Docker volume `edge-data`.

Алгоритм синхронізації

- Крок 1. Edge одержує вимірювання, перевіряє правила та додає його в SQLite.
- Крок 2. Кожні кілька секунд Edge повторно передає пакети до MQTT-теми `cloud/upload`.
- Крок 3. Cloud записує повідомлення в журнал і часову базу InfluxDB.
- Крок 4. Після успішного запису Cloud надсилає `cloud/ack` з ідентифікатором пакета.
- Крок 5. Edge видаляє з локальної черги тільки підтверджений запис.

Дедуплікація

В Cloud журнал має первинний ключ `event_id`. Це не дозволяє подвоїти логічні записи при повторному отриманні пакета. Для часових рядів InfluxDB Cloud використовує однакові теги, час вимірювання і значення полів при повторному надсиланні.

Сценарій перевірки

```
docker compose stop cloud-connector

docker compose logs --tail=15 edge-processor

docker compose start cloud-connector

docker compose logs --tail=15 cloud-connector
```

Очікується, що Edge продовжить обробку датчиків, а кількість записів у `pending` під час відключення зростатиме. Після відновлення Cloud та отримання підтверджень черга поступово скорочуватиметься. Цю перевірку потрібно провести на Mac і показати у відео.

7. Cloud Integration та зберігання

Компонент `cloud/connector.py` реалізує Flask API та клієнт MQTT. Дані з Edge надходять у тему `cloud/upload`. Cloud перевіряє їх, записує у SQLite-журнал, створює точки у вимірюванні `cloud_measurements` InfluxDB і після успіху підтверджує Edge.

Маршрути REST API

HTTP	Маршрут	Призначення
GET	/api/health	Стан Cloud та кількість записів
GET	/api/summary	Поточні показники та статистика
GET	/api/history	Останні вимірювання
GET	/api/aggregate	Групування за хвилиною, годиною, днем
GET	/api/alerts	Останні аномалії
GET	/api/actions	Підтвердження виконаних команд
POST	/api/commands	Відправлення команди до Edge

Аналітичні функції

Модуль `cloud/analytics.py` обчислює середнє, мінімальне та максимальне значення кожного сенсора. Функція `aggregate()` групує історію за хвилинами, годинами або днями та повертає кількість вимірювань для кожного періоду.

Двосторонній обмін

Після натискання кнопки на вебпанелі браузер надсилає HTTP POST на `/api/commands`. Cloud публікує повідомлення у MQTT-тему `cloud/commands`. Edge виконує команду, після чого відповідає у `edge/actions`. Cloud додає результат до журналу, а Dashboard показує останні виконані дії.

8. Dashboard та користувацький інтерфейс

Вебпанель реалізовано без додаткових JavaScript-бібліотек. Nginx на порту 8080 віддає HTML, CSS та JS і проксує запити `/api/` до Flask. Дані оновлюються кожні три секунди через HTTP polling. Графік будується SVG-лінією для останніх 120 значень температури.

Елементи панелі

- Чотири показники: синхронізовані записи, кількість попереджень, затримка Edge та стан системи.
- Шість карток із поточними значеннями датчиків та одиницями вимірювання.
- Графік зміни температури підшипника за останні вимірювання.
- Журнал аномалій із назвою датчика, порогом, дією та часом.
- Кнопки зупинки лінії, запуску лінії, вмикання й вимикання охолодження.
- Підтверджені команди та повідомлення про втрату з'єднання з Cloud.

Схема екрану

Блок 1	Блок 2
Записи Cloud | Аномалії	Latency Edge | Стан
Вібрація | Температура | Струм	Оберти | Шум | Тиск
Графік температури за часом	Автоматичне оновлення 3 с
Журнал аномалій	Керування обладнанням

Для демонстрації достатньо відкрити http://localhost:8080 після запуску Docker. На цій сторінці з'являться реальні дані із симулятора після їх запису до Cloud. Скріншоти працюючої панелі студент повинен отримати на своєму комп'ютері та за потреби додати до звіту.

9. Локальні перевірки програмної логіки

Оскільки Docker Engine у середовищі підготовки відсутній, виконано перевірки функцій Python окремо від контейнерної інфраструктури. Тести запускаються без додаткових пакетів командою `python3 -m unittest discover -s tests -v`.

Результати модульних тестів

Перевірка	Результат
Наявність 6 датчиків	PASS
Генерація 6 категорій несправностей	PASS
Нормальний набір без оповіщень	PASS
Перегрів викликає cooling_on	PASS
Низький тиск викликає critical alert	PASS
Середнє значення за 3 записами	PASS
Агрегація даних за хвилиною	PASS
Обробка некоректного інтервалу	PASS

Додаткова перевірка синтетичного потоку

Локальний симулятор створив 2016 вимірювань, з яких 193 мали штучно встановлену несправність. Прості правила виявили 193 із них, хибних спрацювань у цьому тесті не було. Значення precision та recall становили 100%. Це результат контрольованого генератора, а не оцінка на реальному промисловому обладнанні.

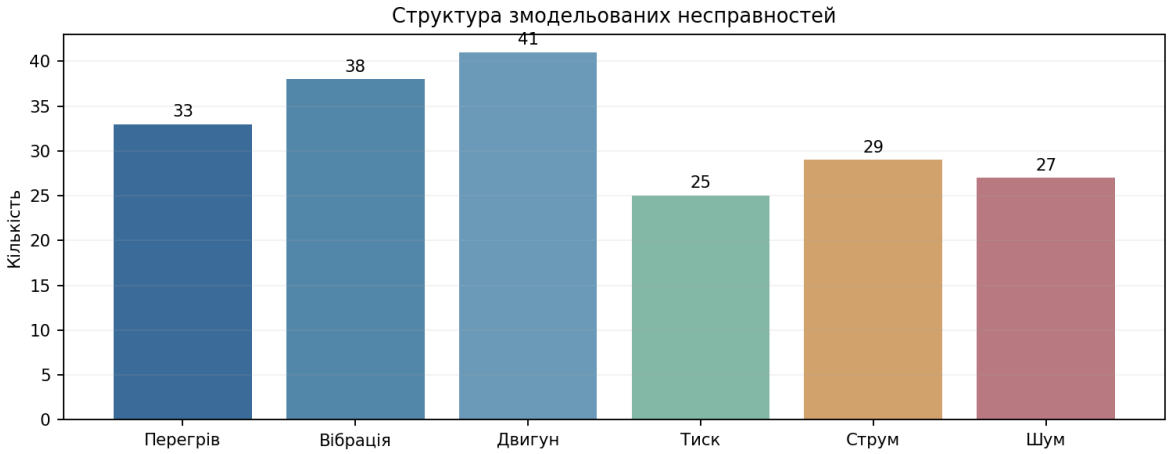


Рисунок 3. Кількість штучних несправностей за типами в локальному тесті.

10. Аналіз затримок, ризиків та обмежень

Виконано локальний замір часу тільки функції `evaluate()` на 2 016 синтетичних пакетах. Середній час становив приблизно 0.0016 мс, а 95-й процентиль приблизно 0.0017 мс. Значення залежать від комп'ютера та не включають MQTT, роботу SQLite, запис в InfluxDB і HTTP.

Що ще необхідно перевірити у Docker

Сценарій	Критерій успіху
Початковий запуск	6 контейнерів у стані Up
MQTT потік	Дані потрапляють від Simulator до Edge
Cloud storage	Лічильник API зростає, історія доступна
Автоматичні дії	При аномалії з'являються EDGE ALERT
Cloud → Edge	Кнопка Dashboard викликає action Edge
Cloud offline	Edge продовжує роботу та накопичує pending
Cloud restart	Черга pending поступово спорожнюється
Latency	Затримка локальних рішень відповідає меті < 100 мс

Можливі недоліки навчального прототипу

- MQTT використовує анонімне підключення. Конфігурація призначена для локальної демонстрації.
- Тестовий Cloud запущено у Docker на тому самому комп'ютері. Це логічне розділення, а не зовнішній інтернет-сервіс.
- При великій кількості накопичених записів повторна синхронізація потребує додаткової оптимізації.
- Правила виявлення штучних несправностей поки не перевірені на даних реальних датчиків.

11. Покрокова перевірка і сценарій захисту

Для відео тривалістю 5–7 хвилин достатньо послідовно показати роботу системи, мережевий обмін і локальну автономність.

Час	Дія
0:00–0:35	Назва практичної, варіант, мета
0:35–1:25	Архітектура та конфігурація Docker Compose
1:25–2:10	docker compose up --build -d, docker compose ps
2:10–3:00	Логи датчиків та Edge, аварійні правила
3:00–3:50	Вебпанель, графік, команди Cloud → Edge
3:50–5:05	Зупинка Cloud, перевірка pending, запуск Cloud
5:05–6:00	API, результати та висновки

Команди автономної перевірки

```
docker compose stop cloud-connector

docker compose logs --tail=15 edge-processor

docker compose start cloud-connector

docker compose logs --tail=15 cloud-connector
```

Дані для внесення після реального запуску

Показник	Значення після запуску
Контейнери у стані Up	_____
Кількість повідомлень у Cloud	_____
Pending під час Cloud offline	_____
Pending після відновлення	_____
Середня затримка Edge	_____
Скріншот Dashboard	Додати за потреби

12. Висновки та рекомендації

У ході роботи розроблено структуру системи Smart Factory Monitoring на основі Docker Compose. Реалізовано симулятор шести датчиків, MQTT-передавання, правила локального Edge реагування, локальний буфер SQLite, Cloud Connector, зберігання в InfluxDB і вебпанель українською мовою.

Виконано вісім модульних тестів Python, які завершилися успішно. Синтетичний потік із 2 016 повідомлень підтвердив роботу заданих порогів для контрольованих аномалій. Повна працездатність MQTT, Docker мереж та відмовостійкість потребують окремого тесту після запуску Docker Desktop на комп'ютері студента.

Переваги обраної архітектури

- Критичні рішення приймаються на Edge без очікування відповіді Cloud.
- Локальний SQLite-буфер зберігає повідомлення під час перерв у роботі Cloud.
- Унікальні event_id і MQTT-підтвердження допомагають уникати втрат і дублювання записів.
- Cloud API дає доступ до поточних даних, історії, агрегатів і команд.

Пропозиції щодо розвитку

- Додати MQTT TLS, паролі та ролі доступу до вебпанелі.
- Перевірити систему на потоці реальних промислових датчиків.
- Додати ML-модель із практичної 9.1 для розпізнавання складніших аномалій.
- Перенести Cloud Connector на окремий сервер та протестувати реальний мережевий розрив.

Комплект для здачі

Архів ZIP з вихідними Python-файлами, Dockerfile, Docker Compose, вебпанеллю, тестами, README, цим звітом у PDF та сценарієм відео. Відео демонстрації необхідно записати окремо після запуску на Mac.

Використані матеріали

Методичні вказівки до практичної роботи 9.2 «IoT Edge Computing + Cloud Integration з Docker». Документація Docker Compose, Eclipse Mosquitto, Flask, InfluxDB та paho-mqtt.